

LLM1 Response:

PDF Processing Logic (Short & Clear)

text

Copy

1. Direct Read (PDFBox) first

└─ Extract text + basic structure

2. Quick Analysis

└─ High text density + clean layout

└─ Detectable tables

└─ Low text / scanned pages

└─ Important images / charts

→ Use extracted text

→ Keep text, mark tables

→ Send those pages to OCR

→ OCR image regions (or later vision)

3. Build intermediate Markdown

└─ From text + OCR results

4. LLM (optional but recommended)

└─ Clean, fix tables, improve structure → Final Markdown

Decision Rules

PDF Type	Primary Path	When to use OCR	When to use LLM
Pure text	Direct Read	Never	Optional (for better structure)
Text + Tables	Direct Read	Rarely	Yes (to format tables properly)
Text + Images	Direct Read	Only on image regions if needed	Optional
Scanned / Image-only	—	Always (all pages)	Yes (clean OCR output)
Mixed (some pages scanned)	Direct + Selective OCR	Only low-text pages	Yes

Core principle:

Direct Read → OCR only when necessary → LLM last (for polishing Markdown).

▼

LLM2 Response:

A page-by-page triage approach is the fastest way to process mixed-content PDFs, as it minimizes expensive OCR calls.

1. Triage (The Initial Read)

Attempt to extract native text from the page using `PDFBox`. Evaluate the result to determine the processing branch.

2. Branch A: Native Text is Present (Text & Tables)

- **Logic:** If text is found, extract it along with its spatial X/Y coordinates.
- **Tables:** Do not use OCR for tables if native text exists. Instead, preserve the horizontal spacing (whitespace) in your extracted string.
- **Routing:** Bypass PaddleOCR entirely to save time.

3. Branch B: Embedded Images or Scanned Pages (Images)

- **Logic:** If the page yields no text, or if `PDFBox` detects specific embedded image blocks within a text page.
- **Routing:** Extract the specific image bytes (or rasterize the whole page if it's a full scan) and send *only* these visual elements concurrently to your PaddleOCR endpoint.

4. Aggregation & LLM Synthesis (Final Step)

- **Merge:** Recombine the native text (from Branch A) and OCR output (from Branch B) sequentially to maintain the document's reading order.
- **LLM Formatting:** Send this assembled payload to the GPT endpoint. Instruct the LLM via the system prompt to interpret spatial gaps as tabular data and output the final result strictly as Markdown tables (`| Column | Column |`) and formatted text.

Why this positioning works for speed:

You only pay the network and latency penalty of PaddleOCR when absolutely necessary (images/scans). Clean text and tables are processed instantly in memory, keeping your response time well under your 30-second limit.

LLM3 Response:

Three tiers, ordered by cost: **direct read ~10 ms**, **OCR ~1-3 s**, **LLM ~2-6 s**. The rule is never escalate a page to the next tier unless the current one demonstrably failed.

Signals to compute per page

Two cheap numbers decide everything:

- `charCount` — from `PDFTextStripper` scoped to that page
- `imageCoverage` — total area of image XObjects ÷ page area

```
java
private double imageCoverage(PDPage page) throws IOException {
    PDResources res = page.getResources();
    double area = 0;
    for (COSName name : res.getXObjectNames()) {
        if (res.isImageXObject(name)) {
            PDImageXObject img = (PDImageXObject) res.getXObject(name);
            area += (double) img.getWidth() * img.getHeight();
        }
    }
    PDRectangle box = page.getMediaBox();
    return area / (box.getWidth() * box.getHeight() * 4); // ~ at 144 DPI equivalent
}
```

Classification

Page profile	<code>charCount</code>	<code>imageCoverage</code>	Route
Text only	≥ 80	< 0.2	Direct read. No OCR, no LLM.
Text + illustration	≥ 80	0.2-0.6	Direct read for text; emit <code>![image]</code> placeholder. OCR the image only if its content matters to you.
Text + tables	≥ 80	any	Direct read , then LLM to rebuild table structure — see below.
Scanned	< 80	> 0.5	OCR. LLM only if confidence is low.
Blank / vector-only	< 80	< 0.5	Skip. Cover pages, separators, pages that are one drawn diagram.
Hybrid	≥ 80	one large image	Direct read the text, OCR the image region separately, merge by y-position.

The last row is the one people miss: a typed form with a photographed signature block is one page needing both routes.

Where the LLM actually earns its cost

Only two jobs, both gated:

1. **Table reconstruction.** `PDFTextStripper` flattens a table into whitespace-separated lines — the cell values survive, the grid does not. Detecting ruling lines yourself is fiddly; handing the flat text to the LLM with "reconstruct as a markdown table, invent nothing" works well. This is the highest-value LLM use in the whole pipeline.
2. **OCR cleanup** when mean confidence drops below ~0.80 — fixing broken hyphenation and line wrapping.

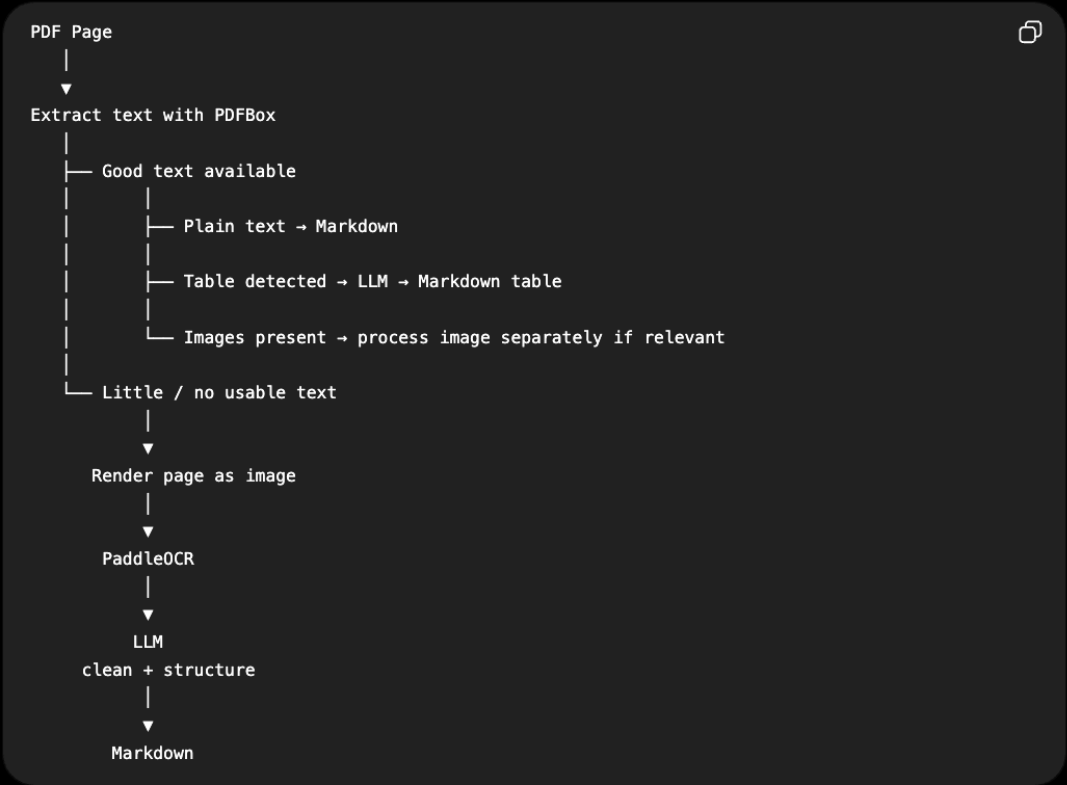
Never on clean native text. The text is already correct; the LLM can only make it worse.

LLM4 Response:

For PDF processing, position the logic **page-by-page**, not for the whole PDF at once.

PDF page type	Direct read	OCR	LLM
Plain digital text	Yes, PDFBox	No	Optional cleanup/understanding
Text + simple table	Yes, PDFBox	Usually no	Yes, to reconstruct table as Markdown
Scanned text page	No usable text	Yes	Yes, cleanup OCR + structure
Image-only page	No	Yes if image contains text	Yes, if image meaning must be understood
Text + embedded image	Read text directly	OCR image only if needed	Combine text + image meaning
Complex table / forms	Try direct extraction first	OCR if extraction is poor	Yes, reconstruct structure

Recommended decision flow:



The core rule should be:

Direct extraction first → OCR only when required → LLM mainly for understanding, structure and Markdown reconstruction.

Avoid sending every PDF page through OCR or LLM; that will increase latency and cost significantly.